GamutX

Error & Exception Handling

Python Basics

GamutX

🎯 Introduction to Error

- What is an Error
- Common Built-in Errors

GamutX

What is an Error?

- An **error** is a problem in the **syntax or structure** of the code.
- It happens before the program even starts running.
- It prevents the code from running at all.

Example

```
print("Hello"
```

❌ Missing closing parenthesis

What Happens?

- Python will not run the program.
- It will show a **SyntaxError**, like this:

```
  File "c:\Users\pralhad\Desktop\New folder\t.py", line 1
    print("hello
                ^
SyntaxError: EOL while scanning string literal
```

EOL = End of Line — Python reached the end but **expected a closing) or ".**

Key Points

- Errors are usually related to typos, missing symbols, or incorrect code structure.
- Common error types include:
  - SyntaxError – bad code structure
  - IndentationError – incorrect spacing
  - NameError – using a variable that hasn't been defined

GamutX

Common Built-in Errors

Python has many types of built-in errors.

Error Type	When It Happens	Example Code
• **SyntaxError**	→ Code doesn't follow Python’s grammar rules	→ **if x == 5 (missing :)**
• **Logical Error**	→ Code runs but produces the wrong result	→ **total = price - tax instead of +**
• **NameError**	→ A variable or function is used before being defined	→ **print(score)**
• **TypeError**	→ An operation uses incompatible types	→ **"hello" + 5**
• **IndexError**	→ A list index is out of range	→ **my_list[10] in a list of 3 items**
• **AttributeError**	→ An object doesn’t have the attribute or method you called	→ **"text".push()**
• **IndentationError**	→ Code is not properly aligned	→ **def func():\nprint("Hi")**

GamutX

🎯 Types of Errors in Python

- Syntax Errors
- Logical Errors
- Runtime Errors
- Type Errors
- Name Errors
- Index Errors
- Attribute Errors
- Indentation Errors

GamutX

Types of Errors in Python

- In programming, errors can happen for many reasons.
- Some errors stop the program before it even runs, and some occur while the program is running.
- **In Python, the most common types of errors include:**

  - Syntax Errors
  - Logical Errors
  - Runtime Errors
    - Type Errors
    - Name Errors
    - Index Errors
    - Attribute Errors
  - Indentation Errors

Syntax Errors

GamutX

- A **syntax error** happens when **you break a rule of the Python language**
- like **forgetting a colon, using wrong punctuation, or missing parentheses.**

Baisc Structure

```
if x > 5
    print("x is greater than 5")
```

missing colon :

Output

```
Traceback (most recent call last):
  File "<main.py>", line 1
    if x > 5
           ^
SyntaxError: expected ':'
```

When It Happens:

- **Before** the program starts **(compile-time)**
- Python stops and refuses to run the program

Logical Errors

GamutX

- A **logical error** happens when the code runs but doesn’t behave as expected due to a **flaw in the logic or reasoning behind the code.**
- The program doesn't crash, but the **result is incorrect.**

🚫 Incorrect

```
def area(length, width):
    return length + width

print(area(5, 3)) # 8
```

Using + instead of *

✅ Correct

```
def area(length, width):
    return length * width

print (area (5, 3)) # 15
```

✅ Correct logic

- But that’s not the area of a rectangle

When It Happens:

- **Run-time:** The program runs, but produces incorrect or unexpected results.

GamutX

Runtime Errors

- A **runtime error** happens **while the program is running.**
- The code might look correct, but when you actually run it, something goes wrong.

Division by Zero

```
x = 10 / 0
```

❌ **Cannot divide by zero**

Output:

```
Traceback (most recent call last):
  File "<main.py>", line 1, in <module>
ZeroDivisionError: division by zero
```

🔍 Why This Happens

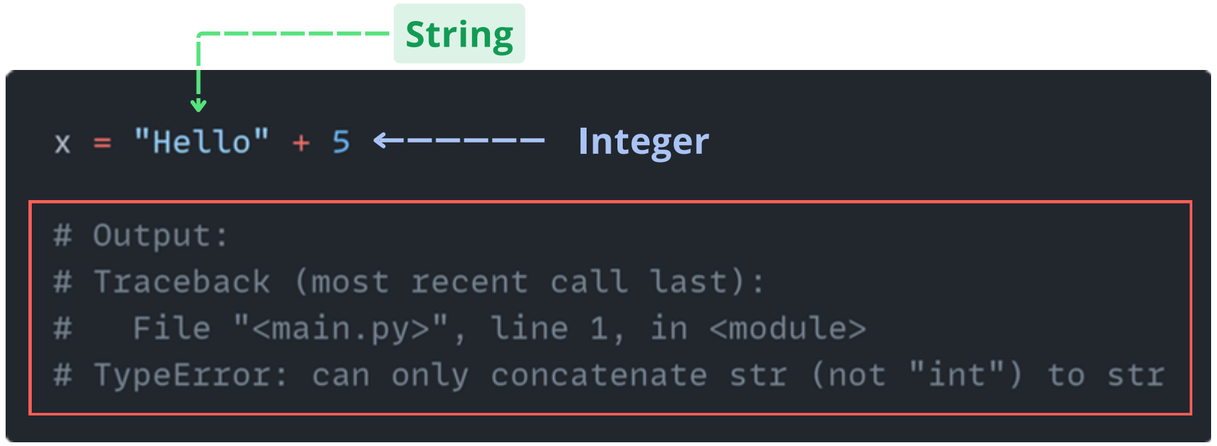
- In this case, you're trying to **divide 10 by 0.**
- In **math, this is not allowed**
- Python stops and **raises a ZeroDivisionError**

GamutX

Type Errors

- A **type error** happens when an **operation is performed on incompatible data types.**
- **Example: trying to add a string and a number**

Add a string and an integer



🔍 Why This Happens:

- Python doesn't know **how to add:**
  - **"Hello" (string)**
  - **5 (integer)**
- So it **throws a TypeError**

When It Happens:

- **Runtime** – the code runs, but crashes when it tries the invalid operation

GamutX

Name Error

- A name error occurs when the **program tries to use a variable or function that hasn’t been defined yet.**

If you try to print a variable that hasn’t been defined

```
print(x)  ⟵ x is not defined
#Output:
Traceback (most recent call last):
  File "<main.py>", line 1, in <module>
NameError: name 'x' is not defined
```

- **x was never assigned a value**
- Python doesn't know what x is, so it **throws a NameError**

When It Happens:

- **Runtime** – The code runs, hits an undefined name, and crashes immediately

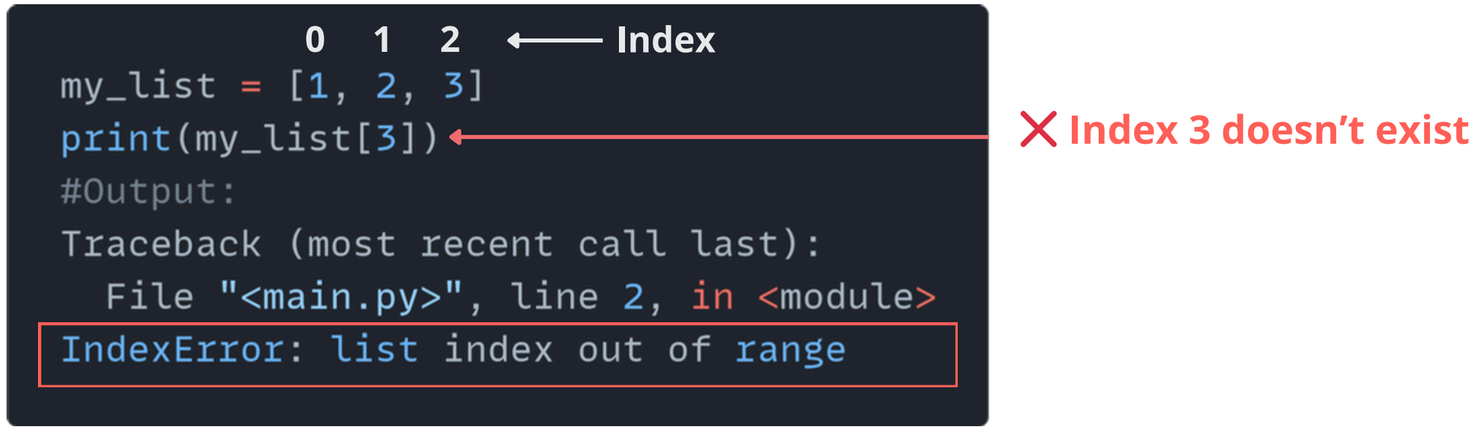
Index Errors

GamutX

- An **index error** occurs **when trying to access an item in a list, array**
- Index is out of range.

Example

if you have a list with 3 elements and we try to access the 4th element



When It Happens:

- **Run-time:** Code runs but crashes when it tries to access a non-existing index

Attribute Errors

GamutX

An **attribute error** happens when you try to **access a method or attribute of an object that doesn't exist.**

Example

- Happens when we try to use a method or attribute that an object doesn't support
- like trying to append to a string

```
my_string = "hello"
my_string.append(" world")
#Output:
Traceback (most recent call last):
  File "<main.py>", line 2, in <module>
AttributeError: 'str' object has no attribute 'append'
```

'str' has no append() method

- append() is valid for lists, not for strings
- Python doesn't recognize the method for this object type → **AttributeError**

When It Happens:

- **Run-time:** code runs, but crashes when an **invalid method is accessed**

GamutX

Indentation Errors

- Python is **strict about indentation**
- Code must be **properly aligned inside functions, loops, and conditionals**

Example

If you forget to indent code inside a function or loop

indented Missing

```
def my_function():
print("Hello")
#Output:
Traceback (most recent call last):
  File "<main.py>", line 2
    print("Hello")  # Indentation Error: this line needs to be indented
    ^^^^^
IndentationError: expected an indented block after function definition on line 1
```

- Python expects the **line after def to be indented**
- **No indentation = IndentationError**

GamutX

Summary of Types of Errors

Errors	Description
• Syntax Errors	→ Mistakes in the code’s grammar or structure.
• Logical Errors	→ Code runs but gives incorrect results due to flawed logic.
• Runtime Errors	→ Program crashes due to unexpected events (e.g., division by zero).
• Type Errors	→ Operations on incompatible data types.
• Name Errors	→ Using undefined variables or functions.
• Index Errors	→ Accessing out-of-range elements in a list or array.
• Attribute Errors	→ Accessing invalid methods or attributes of objects.
• Indentation Errors	→ Incorrect indentation in the code (specific to Python).

GamutX

🎯 Exception Handling

- What is an Exception?
- Common Causes of Exceptions
- Common Built-in Exceptions
- Key Differences Between Errors and Exceptions
- What If We Don’t Handle Exceptions?
- How to Handle It
- Try and Except Block
- Multiple except Blocks
- Catching All Exceptions
- Why It's Important to Handle Exceptions

GamutX

What is an Exception?

- An **exception** is an error that **happens while the program is running.**
- It stops your program from continuing — unless you handle it.

Real-World Analogy:

- Imagine you're **following a recipe.**
- Halfway through, you realize you're **out of eggs.**
- Now you must decide:
  - ❌ **Stop cooking entirely (crash).**
  - ✅ **Find a workaround, like substituting with something else (handle the error).**

That’s what **exception handling** is about: **dealing with unexpected issues gracefully.**

Common Scenarios for Exceptions

GamutX

Here are a few **typical situations that can raise exceptions:**

Scenario	Example
• **Dividing by zero**	→ **10 / 0**
• **Invalid input**	→ **int("abc")**
• **Accessing a missing list item**	→ **my_list[10] when list has only 3 items**
• **Accessing a missing dictionary key**	→ **my_dict["missing_key"]**
• **File not found**	→ **open("missing_file.txt")**

Common Python Built-in Exceptions

GamutX

Python has **many built-in exception** to handle those scenarios.

Human	Python	
Scenario	**Exception Name**	**What Triggers It**
• **Dividing by zero**	• **ZeroDivisionError**	→ **Dividing a number by zero**
• **Invalid input**	• **ValueError**	→ **Passing a bad value to a function**
• **Accessing a missing list item**	• **TypeError**	→ **Using the wrong data type in an operation**
• **Accessing a missing dictionary key**	• **IndexError**	→ **Accessing a list index that doesn't exist**
• **File not found**	• **KeyError**	→ **Accessing a non-existent dictionary key**

GamutX

How to Handle It

Try and Except Block

- The **try** and **except** block is the most basic and commonly used way to **handle errors** (**exceptions**) in Python.
- **It helps you:**
  - Catch errors while the program is running.
  - Show a helpful message to the user.
  - Prevent your program from crashing.

Baisc Structure

```
try:
    # Code that might cause an error
    risky_operation()
except ExceptionType:
    # Code that runs if an error happens
    handle_the_error()
```

Code that might fail

Code to run if an error happens

- catch specific error types like
  - ZeroDivisionError, TypeError, etc

Real World Example

GamutX

We use **try** and **except** to **catch and manage errors**, so program can keep running or fail gracefully.

```
print("Before error")
try:
    x = 10 / 0
except ZeroDivisionError:
    print("Handled the error.")
print("After error")
```

raise ZeroDivisionError

Output:

```
Before error
Handled the error.
After error
```

This avoids crashing and lets the program continue running safely

GamutX

Simple Calculator (Division)

- Let’s say you're asking the user to enter two numbers and then divide them.
- But what i**f the user enters 0 as the second number?**
- That would cause a **ZeroDivisionError**

```
try:
    num1 = int(input("Enter the first number: "))
    num2 = int(input("Enter the second number: "))
    result = num1 / num2
    print(f"Result: {result}")
except ZeroDivisionError:
    print("You cannot divide by zero!")

# output :
# Enter the first number: 2
# Enter the second number: 0
# You cannot divide by zero!
```

raise ZeroDivisionError if user provide 2^{nd} number 0

Concept Practice Task

GamutX

File Not Found Error

```
try:
    with open("non_existent_file.txt", "r") as file:
        content = file.read()
except FileNotFoundError:
    print("Error: The file does not exist.")

# Output:
# Error: The file does not exist.
```

Handling Index Errors

```
my_list = [1, 2, 3]

try:
    print(my_list[5])
except IndexError:
    print("Error: Index out of range.")

# Output:
# Error: Index out of range.
```

Handling Type Errors

```
try:
    result = '5' + 5
except TypeError:
    print("Error: Cannot combine different data types.")

# Output:
# Error: Cannot combine different data types.
```

Handling Attribute Errors

```
my_list = [1, 2, 3]

try:
    my_list.append('4')
    print(my_list.size())
except AttributeError:
    print("Error: 'list' object has no attribute 'size'.")

# Output:
# Error: 'list' object has no attribute 'size'.
```

Concept Practice Task

GamutX

Handling NameErrors

```
def example_function():
    try:
        print(value)
    except NameError:
        print("Error: Local variable referenced before assignment.")

example_function()

# Output: # Error: Local variable referenced before assignment.
```

Handling IOError

```
try:
    file = open("read_only_file.txt", "w")
    file.write("Hello, world!")
except IOError:
    print("Error: Cannot write to read-only file.")

# Output:
# Error: Cannot write to read-only file.
```

Handling Key Errors in Dictionaries

```
my_dict = {'name': 'Alice', 'age': 30}

try:
    print(my_dict['address'])
except KeyError:
    print("Error: Key not found in the dictionary.")

# Output:
# Error: Key not found in the dictionary.
```

IndentationError

```
def demonstrate_potential_error():
    if True:
    print("This line is correctly indented.")

try:
    demonstrate_potential_error()
except IndentationError:
    print("Error: Indentation error encountered. Please check your indentation.")
```

GamutX

Multiple except Blocks

- Sometimes in a program, **more than one kind of error** can happen.
- In these situations, **we can use multiple except blocks to handle each error in its own way.**

Let’s say we’re building a basic calculator that divides two numbers.

Now, two things might go wrong:

- Two possible issues:
- ❌ **Dividing by zero → ZeroDivisionError**
- ❌ **Entering a letter instead of a number → ValueError**

Baisc Structure

```
try:
    # code that might raise different types of exceptions
except ErrorType1:
    # handle the first type of error
except ErrorType2:
    # handle the second type of error
```

GamutX

Handling ZeroDivisionError and ValueError

```
try:
    num1 = int(input("Enter the first number: "))
    num2 = int(input("Enter the second number: "))
    result = num1 / num2
    print(f"Result: {result}")
except ZeroDivisionError:
    print("Error: You cannot divide by zero!")
except ValueError:
    print("Error: Invalid input, please enter numbers!")
```

Handle divide by zero error

Handle invalid input (like letter)

Case 1: Dividing by zero

```
Enter the first number: 5
Enter the second number: 0
Error: You cannot divide by zero!
```

Case 2: Entering non-numeric input

```
Enter the first number: 2
Enter the second number: d
Error: Invalid input, please enter numbers!
```

Concept Practice Task

GamutX

Handling Value and Zero Division Errors

```
try:
    value = int(input("Enter a number: "))
    result = 10 / value
    print("Result:", result)

except ValueError:
    print("Please enter a valid integer.")

except ZeroDivisionError:
    print("Cannot divide by zero.")

# Output:
# If user inputs "a", it prints: Please enter a valid integer.
# If user inputs "0", it prints: Cannot divide by zero.
```

File Operations with Error Handling

```
try:
    filename = input("Enter the filename to read: ")
    with open(filename, 'r') as file:
        content = file.read()
        print(content)

except FileNotFoundError:
    print("The file does not exist.")

except IOError:
    print("An error occurred while reading the file.")

# Output:
# If user inputs a non-existent filename, it prints: The file does not exist.
```

Concept Practice Task

GamutX

Arithmetic Operations with Hardcoded Values

```
try:
    values = [10, 20, 0]  # Hardcoded values
    for value in values:
        result = 100 / value
        print(f"Result of division by {value}: {result}")

except ZeroDivisionError:
    print("Cannot divide by zero.")

# Output:
# Result of division by 10: 10.0
# Result of division by 20: 5.0
# Cannot divide by zero.
```

Handling Index and Key Errors

```
my_list = [1, 2, 3]
my_dict = {'a': 1, 'b': 2}

try:
    index = int(input("Enter an index for the list: "))
    print("List value:", my_list[index])

    key = input("Enter a key for the dictionary: ")
    print("Dictionary value:", my_dict[key])

except ValueError:
    print("Please enter a valid integer index.")

except IndexError:
    print("Index out of range for the list.")

except KeyError:
    print("Key not found in the dictionary.")

# Output:
# If user inputs a non-integer for index, it prints: Please enter a valid integer index.
# If user inputs an index that's out of range, it prints: Index out of range for the list.
# If user inputs a key that doesn't exist in the dictionary, it prints: Key not found in the dictionary.
```

Concept Practice Task

GamutX

Handling Multiple Input Errors

```
try:
    number1 = float(input("Enter the first number: "))
    number2 = float(input("Enter the second number: "))
    quotient = number1 / number2
    print("Quotient:", quotient)

except ValueError:
    print("Invalid input. Please enter numeric values.")

except ZeroDivisionError:
    print("Division by zero is not allowed.")

# Output:
# If user inputs a non-numeric value, it prints: Invalid input. Please enter numeric values.
# If user inputs 0 as the second number, it prints: Division by zero is not allowed.
```

GamutX

Catching All Exceptions

- You can **catch all types of exceptions by using except** without specifying an exception type.
- This can be useful when you're not sure what type of exception might occur.

Baisc Structure

```
try:
    num1 = int(input("Enter the first number: "))
    num2 = int(input("Enter the second number: "))
    result = num1 / num2
    print(f"Result: {result}")
except:
    print("An error occurred.")

# Output:
# Enter the first number: 2
# Enter the second number: 0
# An error occurred.
```

Catches all Exceptions

- This block will catch any exception
  - **ZeroDivisionError, ValueError, or any other error**

Useful when:

- We don't know **what error might occur**
- We want to avoid crashing the program
- **Drawback**: We lose error detail, which makes debugging harder

GamutX

Catching All Exception Vs Catching Specific Exception

Catching All Exceptions

```
try:
    with open("data.txt") as file:
        content = file.read()
except Exception as e:
    print("Something went wrong:", e)
```

catches everything!

including errors we might not expect or want to silently ignore.

Catching Specific Exceptions

```
try:
    with open("data.txt") as file:
        content = file.read()
except FileNotFoundError:
    print("File not found. Please check the filename.")
except PermissionError:
    print("You don't have permission to read the file.")
```

Handles known issue.

✅ **Handles only expected issues clearly and safely.**

What If We Don’t Handle Exceptions?

GamutX

Let’s look at what happens **if you don’t** handle exceptions.

```
print("Before error")
x = 10 / 0
print("After error")
```

It raise ZeroDivisionError

Never Run code

Output:

```
Before error
ZeroDivisionError: division by zero
```

- In a **small script, this might be okay.**

🔍 Real-World Tip:

> ⚠️ In a VFX tool or pipeline, not handling errors can break the whole process

GamutX

Why It's Important to Handle Exceptions

In a real production environment, here's why handling errors is important:

1. Avoid Program Crashes

- Keeps tools stable even if something unexpected happens.
- **Example**: Asset manager tool doesn't crash when an asset is missing.

2. Provide User-Friendly Messages

- Most users don't want (or need) to see a full traceback.
- Clear messages improve usability and reduce frustration.

Example:

```
try:
    number = int(input("Enter a number: "))
except ValueError:
      print("Please enter a valid number.")
```

Provide User Friendly Feedback

3. Allow Recovery or Alternatives

- if a file isn't found, you can create it or ask for another file.

Example:

```
try:
    with open("config.txt") as f:
        config = f.read()
except FileNotFoundError:
    print("Config file not found. Creating default config.")
    config = "default_settings=1"
```

If Config.txt not found this will Apply

- If **config.txt** is missing, the program **doesn’t crash.**
- **It uses a fallback** — which keeps things running.

Differences Between Errors and Exceptions

GamutX

Feature	Error	Exception
When it occurs	→ Before running (at compile time)	→ **While the program is running**
Can it be handled?	→ ❌ No – the program won't run	→ ✅ **Yes – can be caught and handled**
Example	→ Syntax mistake	→ **Dividing by zero, file not found**

Real-Life Analogies

Syntax Error (Type of Error)

- **Analogy:**
  - You’re reading a sign that’s written with bad grammar or spelling:
    - **"Pleze do not enter thiss buildin"**
    - **You struggle to understand it — so you stop reading.**

💡 This is like trying to read a message that’s written wrong. You never get to the actual content.

Runtime Error (Exception)

- **Analogy:**
  - You’re driving and suddenly your car runs out of gas.
    - **You don’t abandon the trip.**
    - **You pause, get help or gas, and continue driving.**

💡 This is an **exception** — something unexpected, but you can deal with it.

GamutX

🎯 Common Mistakes in Exception Handling

- Catching All Exceptions Too Broadly
- Ignoring the Exception
- Overusing Exception Handling

Catching All Exceptions Too Broadly

GamutX

- Using a **generic except** block without **specifying the type of error can catch things you didn't intend to.**
- This **makes debugging hard and hides real problems** in your code.

🚫 Bad Practice:

```
try:
    with open("non_existent_file.txt", 'r') as file:
        content = file.read()
    num1 = 10
    num2 = 0
    result = num1 / num2

    my_list = [1, 2, 3]
    print(my_list[5])

except Exception:
    print("An error occurred.")

# Output : # An error occurred.
```

Catches Generic Exceptions

What's Wrong?

You don't know which error occurred

- Could be:
  - **File not found**
  - **Division by zero**
  - **Index out of range**
- Also catches system-level exceptions (e.g. **KeyboardInterrupt**)

GamutX

✅ Best Practice:

```
try:
    with open("non_existent_file.txt", 'r') as file:
        content = file.read()
    num1 = 10
    num2 = 0
    result = num1 / num2
    my_list = [1, 2, 3]
    print(my_list[5])

except FileNotFoundError:
    print("Error: The file does not exist.")
except ZeroDivisionError:
    print("Error: Cannot divide by zero.")
except IndexError:
    print("Error: Index out of range.")
except Exception as e:
    print(f"An unexpected error occurred: {e}")

# Output:
# Error: The file does not exist.
```

→ Handles only relevant errors — clearly and safely

GamutX

Ignoring the Exception

- Sometimes people **use pass inside an except block to simply ignore the error.**
- This prevents the program from crashing,
- but i**t hides the problem and gives no feedback to the user.**

🚫 Bad Practice:

```
try:
    value = int(input("Enter a number: "))
except ValueError:
    pass  # Ignoring the error
```

What's Wrong?

- If the user types something that isn't a number,
- the program just does nothing.
- The user won't know that something went wrong.

✅ Best Practice:

```
try:
    value = int(input("Enter a number: "))
except ValueError:
    print("That's not a valid number. Please try again.")
```

This way, the user knows what happened and can fix it.

GamutX

Overusing Exception Handling

- **Using exceptions to control normal program flow is not a good habit.**
- For example, using try-except to check if a file exists

🚫 Bad Practice:

```
try:
    with open("nonexistent_file.txt") as f:
        data = f.read()
except FileNotFoundError:
    data = "Default data"
```

Relies on exception to control normal logic

What's Wrong?

- The code depends on an exception to work.
- It's harder to understand and maintain.
- You could check if the file exists before trying to open it.

✅ Best Practice:

Use exception handling only for exceptional cases, not regular logic.

```
def read_file(file_path):
    try:
        with open(file_path, 'r') as f:
            data = f.read()
            return data
    except FileNotFoundError:
        return "Default data"

file_name = "nonexistent_file.txt"
data = read_file(file_name)
print(data)

# Output:
# Default data
```

Fallback returned only if file is missing

Now the file reading is neatly handled inside a function. If the file isn't found, it gives a clear fallback.

GamutX

Thank You